

> **Ficha Prática Nº11 (React – Lógica das Cartas e Pontuação)**

Durante as últimas aulas práticas, foram introduzidos vários conceitos para implementação do Jogo de Memória. Entre eles a criação de **componentes** React, **props** e **Hooks**, como o **useState** e o **useEffect**, que permitem a criação de variáveis de estado e efetuar a gestão de “*side-effects*”, no ciclo de vida do componente. Esta ficha, pretende dar continuidade e concluir o Jogo de Memória em React, aplicando esses conceitos e implementar a restante lógica do jogo, nomeadamente, todo o comportamento das cartas quando clicadas assim como a atualização da pontuação.

> **Preparação do ambiente**

- a. Descompacte o ficheiro **ficha11.zip** e inicie o *Visual Studio Code*.
- b. Abra a **pasta de trabalho no workspace**.



Os alunos que concluíram a resolução da ficha anterior, devem continuar nesse trabalho.

Os restantes alunos podem resolver esta ficha prática, tendo como base o código fornecido juntamente com esta ficha (Ficha 11), no entanto, devem ter em consideração, que apenas está disponível a versão para implementação com o nível básico. Recomenda-se a resolução das fichas anteriores para quem pretender o jogo na sua totalidade.

- c. No terminal, digite os seguintes comandos:
 - **npm install** (para instalar as dependências em falta)
 - **npm start** (para iniciar a aplicação)
- d. Confirme a aplicação no browser, no endereço <http://localhost:3000/>

Parte I – Exercício: Rotação das Cartas e fim de Jogo

Pretende-se que o aluno implemente o código para rotação das cartas existentes no tabuleiro de jogo assim como, para permitir identificar se duas cartas são iguais. Além disso, pretende-se identificar o fim de jogo, o que ocorre quando todos os pares de cartas são encontrados. As figuras seguintes apresentam a interface a implementar.



Figura 1 – Rotação das cartas

1> O primeiro comportamento a implementar refere-se à rotação da carta após ser alvo de um clique. Assim, efetue os seguintes passos:

a. No componente **Card**:

→ Remova a class **flipped**, caso esteja a ser aplicada no elemento **div**, de forma a que o tabuleiro fique com todas as cartas com o aspeto da Figura 2. Da mesma forma que na implementação em JavaScript, ao aplicar a classe **flipped**, estamos a apresentar a imagem do logotipo da carta e não o “card-back”.



Figura 2 - Card-back

→ A partir de agora, a aplicação da class **flipped**, vai depender do *click* efetuado à carta. Para isso, será necessária uma variável de estado. Portanto:

- Declare e inicialize a variável de estado **flipped**, com valor inicial **false**, não esquecendo de efetuar o *import* do *useState*, caso ainda não tenha sido adicionado.
- Adicione o código para que, quando o estado **flipped** for false não seja aplicada esta classe no div, caso contrário (variável de estado **flipped** a **true**) a classe deve ser aplicada.
- Adicione o código necessário, para que quando for efetuado um *clique* sobre a carta, a variável de estado **flipped**, seja true. Note que este comportamento, apenas deverá ser aceite, se o jogo já tiver sido iniciado.
- Confirme o comportamento no browser e clique sobre as cartas.

Implemente o código de forma a que o comportamento da alínea a) seja aplicado apenas quando o jogo já tenha sido iniciado. Para tal recorra à variável de estado *gameStarted*, declarada no App, a qual indica se o jogo já iniciou ou não. O valor desta variável de estado deve ser enviada/recebida pelos(as) atributos/props do App -> gamePanel -> Card, como se representa no diagrama seguinte.

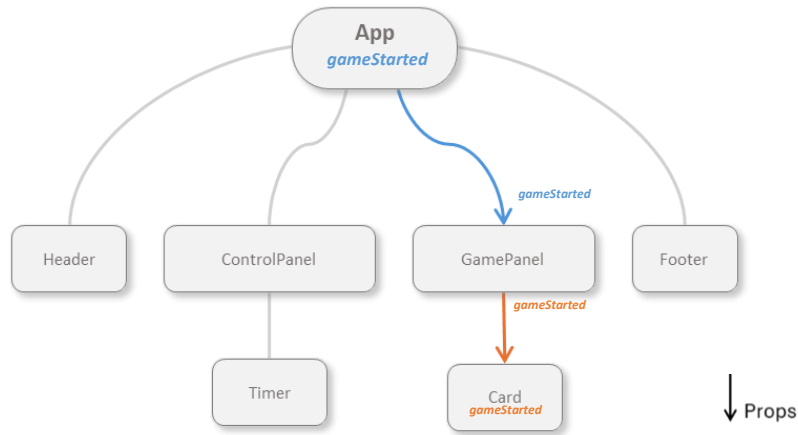


Diagrama 1 - Diagrama de Componentes com gameStarted

- Ao receber as props no objeto **card**, efetue a atribuição, via desestruturação de objeto, ao valor *gameStarted* e *name*, que já está a ser usado no atributo data-logo, do elemento div assim como na definição do valor do atributo src na imagem (card-front).
- Confirme o comportamento no browser e clique sobre as cartas. Como pode verificar, caso termine/inicie o jogo, existem situações em que nem todas as cartas voltam à posição original. Esta situação, será resolvida ao implementar as secções seguintes.

2> Na secção anterior, foi implementada a rotação de uma carta após ser alvo de um clique. Nesta secção, será implementada a verificação, quando duas cartas estão viradas, se formam ou não um par. Para isso, implemente os passos seguintes.

- a.** Para identificar se duas cartas formam par, será aplicado o mesmo raciocínio utilizado na implementação do jogo em JavaScript. Assim, cada uma das cartas clicadas logo após sofrer o evento, será armazenada num array. Depois será efetuada a comparação das cartas, no componente **GamePanel**. Para isso, implemente os seguintes passos:

→ Componente **GamePanel**:

- > Declare o array **flippedCards**. Este array, terá no máximo dois elementos, isto é, as duas cartas clicadas. Este array não necessita de ser uma variável de estado uma vez que a alteração do seu valor não justifica uma renderização automática no componente **GamePanel**.

- > Implemente a função *callback* `handleFlippedCards`:
 - Deve receber por parâmetro o nome da carta clicada e adicioná-lo ao *array* declarado anteriormente.
 - A função, deverá ainda, escrever na consola (apenas para debug) uma mensagem, quando forem adicionadas duas cartas na consola.
- > Especifique o atributo `onFlippedCards` quando invoca o componente `Card`, o qual servirá para enviar a referência da função implementada anteriormente.

→ Componente **Cards**:

- > Receba a propriedade `onFlippedCards`;
- > Invoque a função, sempre que clicar numa carta. Considere, que provocar a rotação da carta, só faz sentido se a carta estiver oculta e o jogo já tiver sido iniciado.

→ Confirme o comportamento no browser e confirme que, quando após clicar duas cartas, é apresentada a mensagem.

b. Neste passo, pretende-se verificar se as duas cartas clicadas são iguais e em caso verdadeiro, as cartas fiquem com aspeto bloqueado, caso contrário, as cartas deixam de estar viradas. Para isso, implemente os passos seguintes.

→ Componente **GamePanel**:

- > Declare e inicialize a variável de estado `matchedCards`, com valor inicial `[]`. Esta variável irá armazenar os pares de cartas iguais que forem sendo encontrados.
- > Implemente a função `processMatchingCards` o qual deverá verificar se as duas cartas no array `flippedCards` são iguais
 - Em **caso de serem iguais**, deverá atualizar a variável de estado `matchedCards` com as duas cartas;
 - Em caso de **não serem iguais**, deverá, por exemplo, atualizar a `matchedCards` com o mesmo estado, para forçar nova renderização do `GamePanel`;
 - *Tal* como efetuado, quando o jogo foi implementado em JavaScript, adicione a função `setTimeout` para implementar o comportamento quando são iguais/diferentes.
 - Depois, remova todas as cartas existentes no array `flippedCards`.
- > Substitua o `console.log` especificado anteriormente na função `handleFlippedCards`, pela invocação da função `processMatchingCards`.
- > Especifique o atributo `matchedCards` quando invoca o componente `Card`, o qual servirá para enviar quais as cartas foram já encontradas (`matchedCards`)

→ Componente **Card**:

- > Receba a propriedade **matchedCards**;
- > Invoque a função **onFlippedCards** sempre que clicar numa carta.
- > Declare e inicialize a variável de estado **matched**, com valor inicial **false**. Esta variável irá armazenar se a carta já foi ou não encontrada.
- > Adicione o seguinte **useEffect**, sempre que o array dos pares de cartas encontradas for atualizado, atualiza o estado *flipped* e *matched* das cartas.

```
useEffect(() => {
  const isMatchedCard = matchedCards.filter(
    (logoName) => logoName === name).length > 0;
  setFlipped(isMatchedCard);
  setMatched(isMatchedCard);
}, [matchedCards, name]);
```

→ Visualize o jogo no browser e confirme que não existem erros até ao momento e que é possível identificar os pares de cartas.

- c.** Pretende-se terminar o jogo quando todas as cartas forem identificadas. Esse comportamento será efetuado através de um **useEffect**, que deverá ser invocado sempre que a variável de estado **matchedCards** for atualizada.

→ Copie o código seguinte para o componente **GamePanel**:

```
useEffect(() => {
  if (matchedCards.length === cards.length && gameStarted)
  {
    onGameStart();
  }
}, [matchedCards, cards, gameStarted, onGameStart]);
```

- > Receba a propriedade **onGameStart** vinda do App de forma a ser possível terminar o jogo.
 - > Visualize o jogo no browser.
- Implemente um **useEffect**, no componente **GamePanel**, de forma a ser invocado sempre que o jogo termina (**!gameStarted**). A variável de estado **matchedCards** deve ser reiniciada com um array vazio.
- Visualize o jogo no browser.

3> Pretende-se agora implementar a pontuação do jogo. O cálculo da pontuação, deverá ser implementado no componente App, no entanto, é invocado a partir do componente GamePanel, na função que compara se as cartas são iguais. Assim, sempre que encontra um par implementa a fórmula de cálculo descrita no caso seguinte, caso as cartas não coincidam, são descontados 5 pontos, não podendo, no entanto, existir pontuações negativas.

Assim, implemente os seguintes passos:

a. Componente App:

- Declare a variável de **estado totalPoints**, com o valor inicial **0**.
- Declare a variável **timer**
 - Esta variável deverá ser atualizada com o valor dos segundos existente no componente **ControlPanel**. Para isso, implemente a função callback **handleTimerApp** e efetue as alterações necessárias nos componentes **App** e **ControlPanel**.
- Copie o seguinte código:

```
const handlePoints = (operacaoSoma = true) => {
  let pointsSum = totalPoints;
  if (operacaoSoma) {
    pointsSum += timer * (cards.length / 2);
  } else {
    pointsSum < 5 ? (pointsSum = 0) : (pointsSum -= 5);
  }
  setTotalPoints(pointsSum);
};
```

- Adicione os atributos de forma a que a função anterior possa ser invocada no componente **GamePanel**, a partir da função **processMatchingCards**.
- Adicione o atributo **totalPoints** quando invoca o Control-Panel, de forma a que possa enviar o cálculo da pontuação para o componente.
- Especifique a variável **totalPoints** a 0, sempre que se inicia novo jogo.

b. Componente Control-Panel:

- Apresente a pontuação obtida por prop.
- Visualize o jogo no browser.

- 4> Pretende-se ainda apresentar a janela modal com a indicação da pontuação obtida sempre que o jogo termine, de forma a ficar com o aspeto da Figura 3.

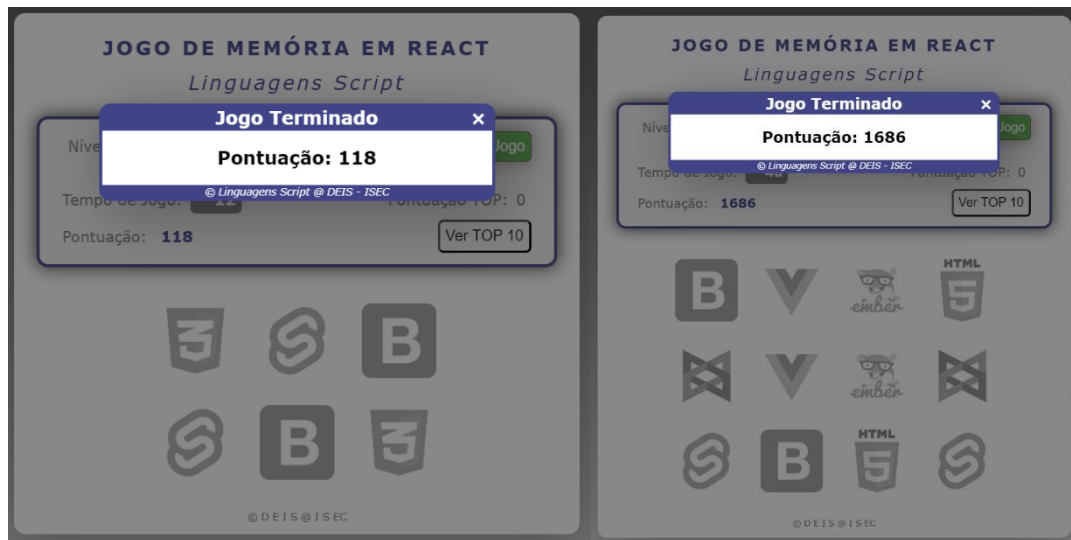


Figura 3 - Janela Modal de fim de jogo

Verifique, na estrutura do projeto, que já existe o componente **GameOverModal**. Este componente, recebe por props os seguintes valores/referências:

- **isOpen**: indica se a variável modal deve ser apresentada;
- **points**: os pontos a serem apresentados;
- **handleClose**: a função *callback* a ser executada após o fecho da janela modal.

Assim, para efetuar o pretendido, implemente os seguintes passos:

a. Componente App:

- Declare a variável de estado **isGameOverModalOpen**, com o valor inicial **false**;
- Implemente a função **handleGameOverModalClose**, *callback function*, que deverá colocar a variável de estado anterior a **false**, e além disso, terminar o jogo.
- Especifique o código seguinte imediatamente antes da invocação do componente **Footer**.

```
<GameOverModal isOpen={isGameOverModalOpen}
                handleClose={handleGameOverModalClose}
                points={totalPoints}
            />
```

- Por fim, implemente a função **handleGameOver**, a ser invocada quando o jogo termina. Esta

função deve colocar a variável de estado **isGameOverModalOpen** com o valor **true**, garantindo dessa forma a janela modal é apresentada.

- b.** De forma a invocar a função **handleGameOver** quando o jogo termina, por terem sido encontradas todas as cartas, no componente **GamePanel**:
 - Substitua, no *useEffect*, a invocação de **onGameStart** por **onGameOver**.
 - Deve garantir que recebe via *props*, o **onGameOver**, o qual contém a referência do **handleGameOver**.
- c.** De forma a invocar a função **handleGameOver** quando o jogo termina, por ter sido esgotado o tempo de jogo, no componente **ControlPanel**:
 - Substitua a invocação de **onGameStart** por **onGameOver**, quando o tempo chegou a 0.
 - Deve garantir que recebe via *props*, o **onGameOver**, o qual contém a referência do **handleGameOver**.
- d.** Confirme o jogo no browser.
- e.** Jogue, depois de encontrar todas as cartas, verifique o que acontece ao *tempo de jogo*, mesmo com a janela modal aberta. Para resolver este problema, será necessário cancelar o temporizador e a única forma de o fazer é por recurso ao id do *interval* gerado no Timer. Assim, no componente **ControlPanel**:
 - A função **handleTimer**, para além de receber os segundos, deve ainda receber o id, para poder cancelar o tempo (deve efetuar as alterações necessárias, de forma a que o componente **Timer**, envie os dois valores).
 - Repare que a forma de cancelar o tempo, na situação em que o jogo termina quando todos os pares de cartas são corretamente identificados, deverá ser efetuada no App ou no GamePanel. Assim, a função *handleTimer*, deverá enviar o id do timer para o componente pai (App).
 - Tendo por base todos os conhecimentos obtidos até ao momento, efetue essas alterações, devendo o código **clearInterval(timerId)** ser invocado na função **handleGameOver**.
- f.** Confirme no browser que o jogo assume o comportamento desejado.

Parte III – Exercício para praticar em casa

Implemente o código necessário para, quando o jogo ainda não tiver iniciado, ser apresentada:

- a.** a mensagem “Clique em iniciar” ao clicar no tabuleiro de jogo.

- b.** A visibilidade da mensagem anterior deve alternar mediante o cliques sucessivos no tabuleiro de jogo. As figuras seguintes apresentam o comportamento desejado:

→ “Clique em Iniciar Jogo” → “” → “Clique em Iniciar Jogo” → “” →

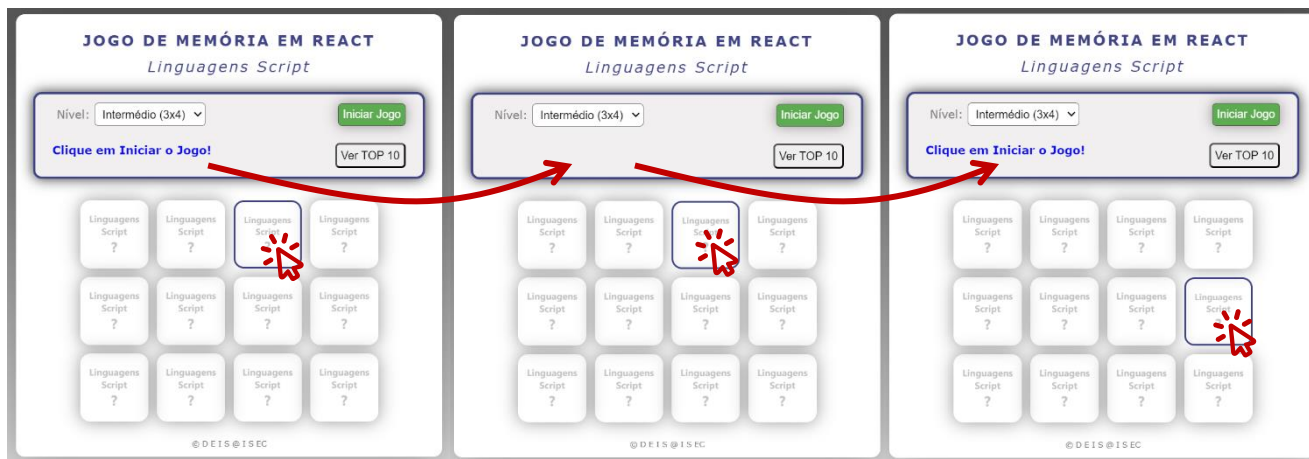


Figura 4 - Cliques no PanelGame sem Iniciar jogo